

Strategy e State

Padrões de Projeto - Aula #08

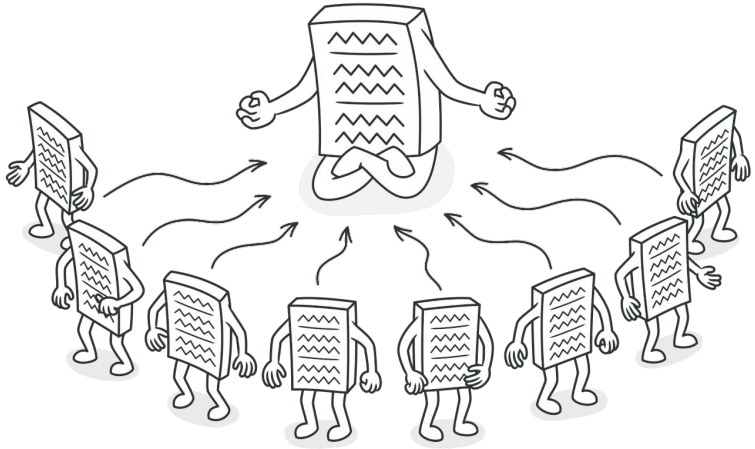
Prof. Dr. Carlos Melo

carlos.melo@upe.br

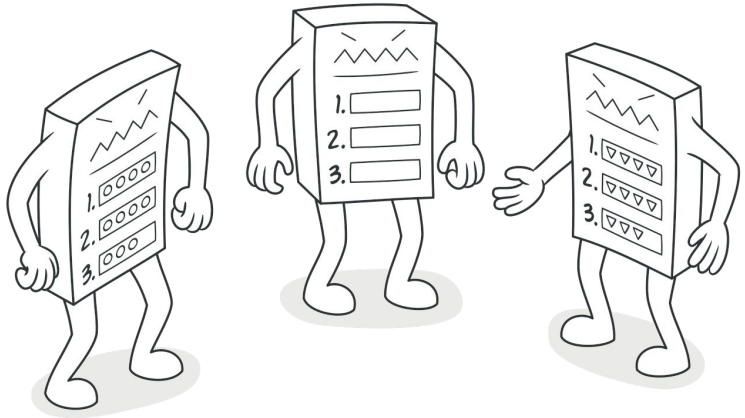
UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Padrões Comportamentais

Strategy
Família de algoritmos intercambiáveis, escolhidos em tempo de execução



State
O objeto muda de comportamento quando seu estado interno muda



- Os dois têm diagramas parecidos, o que muda é a intenção.

Objetivos de aprendizagem

- Implementar *strategies* e saber quando uma função basta no lugar de uma classe
- Identificar uma máquina de estados escondida em `ifs` de algum atributo
- Implementa *State* e *transition* (algo que o Strategy não tem)

Strategy - O Problema

- Temos o UPEMaps que é um app de navegação
- A primeira versão só traçava rotas de carro

```python
class Navegador:
    def construir_rota(self, origem, destino):
        # 100+ linhas de algum algoritmo de menor caminho em grafos
        return ["seguir BR-232", "converter à direita", "chegou"]
```

- Novos requisitos:
  - rotas a pé
  - rotas de transporte público
  - rotas de bicicleta
  - rotas turísticas passando por pontos de interesse

Strategy - O Problema

```python
class Navegador:
    def construir_rota(self, origem, destino, modal):
        if modal == "carro":
            ...   # 100+ linhas
        elif modal == "pe":
            ...   # 100+ linhas
        elif modal == "publico":
            ...   # 100+ linhas
        elif modal == "bicicleta":
            ...   # 100+ linhas
```

- A classe cresceu 300%
  - Qualquer alteração de algoritmo pode demandar alterar uma classe compartilhada por todos os meios de transporte e ocasionar conflitos na hora de algum merge no *`github`*

Strategy - O Problema

- Usar **`if`** e **`else`** não é o problema, mas sim o fato de todos os algoritmos estarem armazenados na mesma classe
  - Se a classe cresce a cada requisito é sinal de que estamos violando o **O** do S**O**LID, uma vez que cada extensão vai exigir edição
  - A classe tem quatro responsabilidades, o que nos leva a viola o **S** do **S**OLID
  - Testar uma rota a pé, requer carregar um objeto **`Navegador`** com todos os algoritmos
  - Todo mundo vai editar o mesmo arquivo
- Além disso, os quatro blocos têm algo em comum:
  - **`rota = algoritmo(origem, destino)`**
  - Mesma assinatura, mesma finalidade, diferentes implementações
  - A isso chamamos de família de algoritmos intercambiáveis

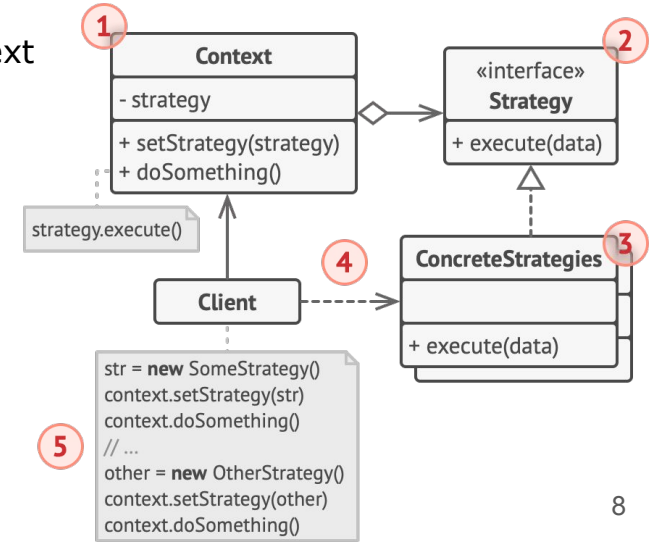
Strategy - Intenção

"Definir uma **família** de algoritmos, **encapsular** cada um deles e torná-los **intercambiáveis** . Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam."

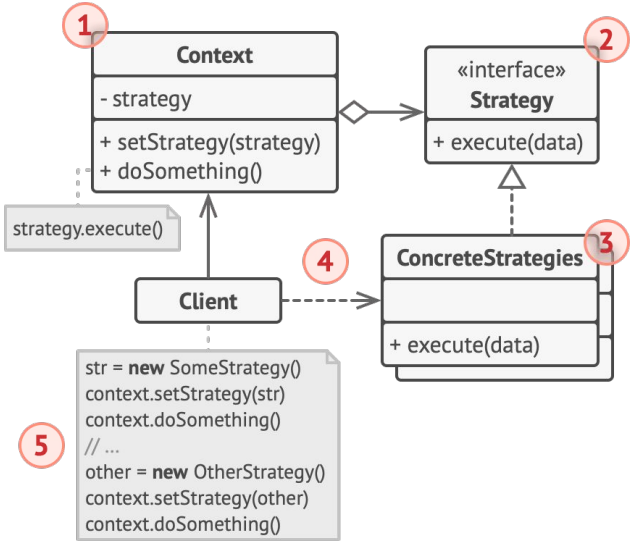
- Padrão comportamental
- **Família**:
  - Algoritmos que fazem a mesma coisa de formas diferentes
- **Encapsular**:
  - Cada um em sua própria classe
- **Intercambiáveis**:
  - O contexto pode ser trocado sem a necessidade de saber qual é qual

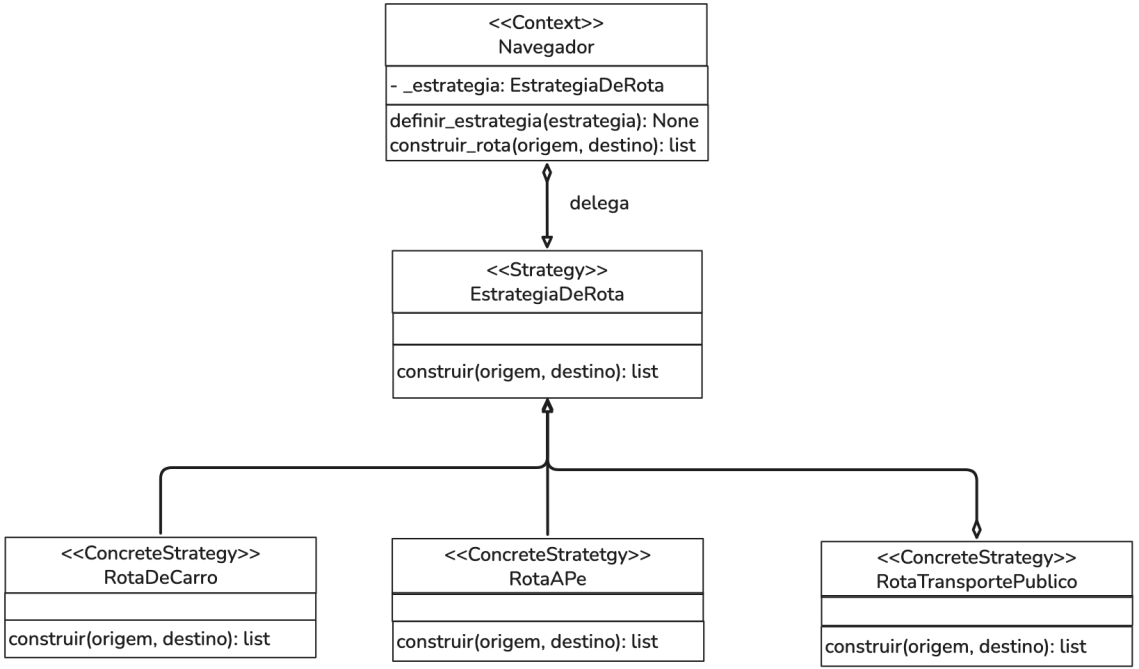
Strategy - Estrutura

1. **Context** mantém uma referência para uma das **ConcreteStrategies** e se comunica com esse objeto através da interface da estratégia
2. A interface da **Strategy** é comum a todas as **ConcreteStrategies**.
3. **ConcreteStrategies** implementam diferentes variações de um algoritmo que **Context** usa.
4. O Context chama o método de execução no objeto strategy ligado cada vez que ele precisa rodar um algoritmo. O contexto não sabe qual tipo de estratégia ele está trabalhando ou como o algoritmo é executado.
5. **Client** cria um objeto Strategy específico e o repassa para o Context



Aplicando ao nosso contexto





Aplicando ao nosso contexto

```python
from abc import ABC, abstractmethod

class EstrategiaDeRota(ABC):
    @abstractmethod
    def construir(self, origem: str, destino: str) -> list[str]: ...


class RotaDeCarro(EstrategiaDeRota):
    def construir(self, origem, destino):
        return [f"Saia de {origem} pela BR-232",
                "Siga por 230 km",
                f"Chegou em {destino} — 3h10, 18 L de combustível"]

class RotaAPe(EstrategiaDeRota):
    def construir(self, origem, destino):
        return [f"Saia de {origem} pela calçada à direita",
                "Atravesse a praça central",
                f"Chegou em {destino} — 47h, evite trechos sem acostamento"]

class RotaTransportePublico(EstrategiaDeRota):
    def construir(self, origem, destino):
        return [f"Terminal de {origem}: ônibus expresso 14:30",
                "Baldeação em Caruaru",
                f"Chegou em {destino} — 4h20, R$ 62,00"]
```

Versão Pythonica do Strategy

- A interface tem um método. Em Python é só uma função.

```python
from typing import Callable

Estrategia = Callable[[str, str], list[str]]

def rota_de_carro(origem, destino):
    return [f"Saia de {origem} pela BR-232", "..."]

def rota_a_pe(origem, destino):
    return [f"Saia de {origem} pela calçada à direita", "..."]


class Navegador:
    def __init__(self, estrategia: Estrategia):
        self._estrategia = estrategia

    def construir_rota(self, origem, destino):
        for passo in self._estrategia(origem, destino):
            print(passo)
```

Versão Pythonica do Strategy

- Use classes quando a estratégia tem estado ou mais de uma operação.
  - Caso contrário, função.
- O partial serve para criar uma nova função fixando antecipadamente alguns valores para os argumentos de uma função já existente.

```
Navegador(rota_de_carro).construir_rota("Garanhuns", "Recife")


from functools import partial
rota_turistica = partial(rota_de_carro, evitar_pedagio=True)
Navegador(rota_turistica).construir_rota("Garanhuns", "Recife")


MODAIS = {"carro": rota_de_carro, "pe": rota_a_pe}
Navegador(MODAIS[escolha_do_usuario]).construir_rota(origem, destino)
```

Prós e Contras do Strategy

Prós

- Troca de algoritmo em tempo de execução
- Isola a implementação do algoritmo do código que o usa
- Prioriza composição
- Novas estratégias entram sem tocar no contexto (S**O**LID)
- Cada algoritmo pode ser testado individualmente sem tocar no contexto

Contras

- Se você tem dois algoritmos que nunca mudam, o padrão é complicação adicional
- O cliente precisa conhecer a estratégia para escolher uma
- Muitas classes pequenas para pouco código

Não use:

- Se tiver um único algoritmo ou quando não precisa realizar a troca em tempo de execução. Muitas vezes o if e else basta

State - O Problema

- Um sistema de publicação de documentos
- Um documento pode estar em três estados:
  - `Rascunho`
  - `Moderação`
  - `Publicado`
- A mesma operação, `publicar()`, tem efeitos diferentes:
  - Em `rascunho` ela encaminha o documento para `moderação`
  - Em `moderação` o documento é `publicado` quando o usuário é `administrador`
  - Em `publicado` o documento é mantido `publicado`
- Na `moderação`, uma tentativa de `publicação` por outro perfil mantém o estado atual

State - Implementação com condicionais

```python
class Documento:
    def __init__(self) -> None:
        self.estado = "rascunho"

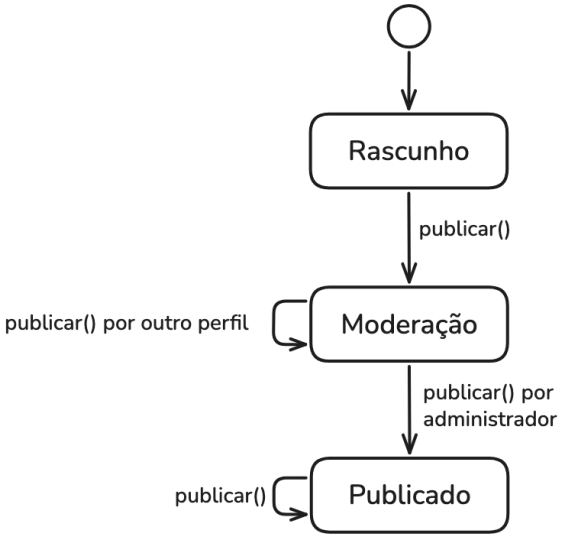
    def publicar(self, perfil: str) -> None:
        if self.estado == "rascunho":
            self.estado = "moderacao"

        elif self.estado == "moderacao":
            if perfil == "administrador":
                self.estado = "publicado"

        elif self.estado == "publicado":
            pass
```

- Essa implementação pode ser suficiente para um caso pequeno
- O problema aparece quando:
  - Várias operações repetem verificações do estado atual
  - Novos estados exigem alterações em diferentes métodos
  - As regras de comportamento e de transição ficam espalhadas pela classe

State - estados e transições do documento



- O documento mantém um estado ativo por vez
- A operação `publicar()` depende do estado atual e do perfil recebido
- As setas que retornam ao mesmo estado representam tentativas que mantêm esse estado

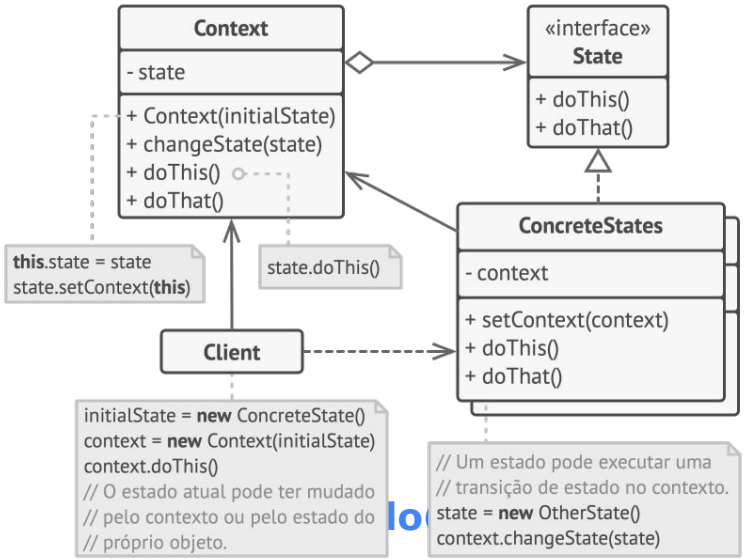
State - ideia e intenção

"Permitir que um objeto altere seu comportamento conforme seu estado interno muda"

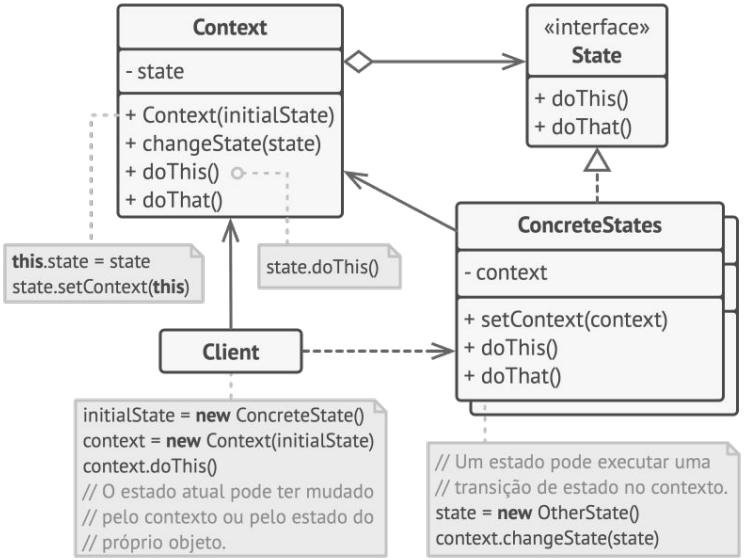
- No documento:
  - Cada estado terá uma classe própria
  - Todas as classes de estado oferecerão a operação `publicar()`
  - O `Documento` manterá uma referência ao estado ativo e delegará a operação
  - Uma transição substituirá o objeto de estado associado ao documento
- As regras de publicação ficam nas classes Rascunho, Moderacao e Publicado

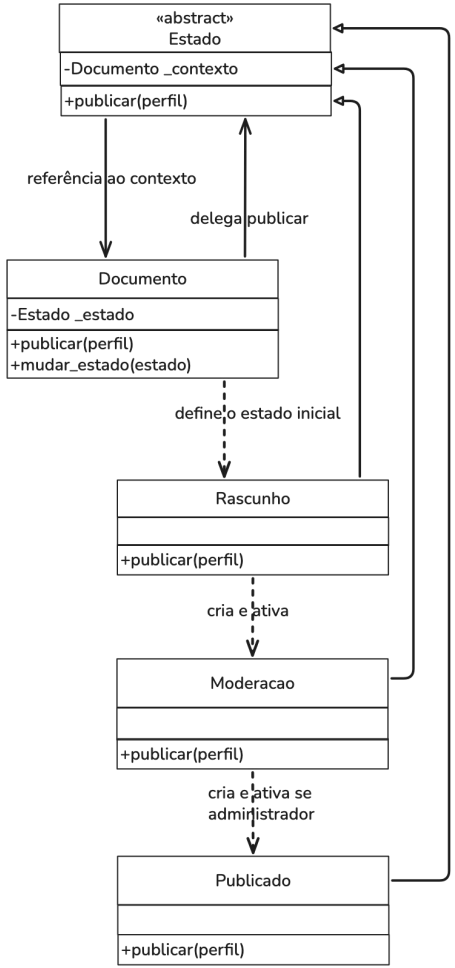
State - Estrutura

1. O **Context** mantém o estado ativo, delega operações e oferece uma operação para substituí-lo
2. **State** declara o contrato dos comportamentos que variam
3. **ConcreteStates** implementam esses comportamentos e podem manter uma referência ao contexto
4. Transições determinam o contexto e os **ConcreteStates** podem determinar a troca do estado ativo.



Aplicando ao nosso contexto





Aplicando ao nosso contexto - State

```python
from abc import ABC, abstractmethod


class Estado(ABC):
    def __init__(self, contexto: "Documento") -> None:
        self._contexto = contexto

    @abstractmethod
    def publicar(self, perfil: str) -> None:
        pass
```

- `_estado` guarda o objeto responsável pelo comportamento atual
- A propriedade estado permite consultar o nome desse estado
- `mudar_estado()` substitui o estado ativo
- `publicar()` delega a decisão ao estado atual

Aplicando ao nosso contexto - Context

```python
class Documento:
    def __init__(self) -> None:
        self._estado: Estado = Rascunho(self)

    @property
    def estado(self) -> str:
        return type(self._estado).__name__

    def mudar_estado(self, estado: Estado) -> None:
        self._estado = estado

    def publicar(self, perfil: str) -> None:
        self._estado.publicar(perfil)
```

- `_estado` guarda o objeto responsável pelo comportamento atual
- A propriedade estado permite consultar o nome desse estado
- `mudar_estado()` substitui o estado ativo
- `publicar()` delega a decisão ao estado atual

Aplicando ao nosso contexto - Concrete State

```python
class Rascunho(Estado):
    def publicar(self, perfil: str) -> None:
        self._contexto.mudar_estado(Moderacao(self._contexto))


class Moderacao(Estado):
    def publicar(self, perfil: str) -> None:
        if perfil == "administrador":
            self._contexto.mudar_estado(Publicado(self._contexto))


class Publicado(Estado):
    def publicar(self, perfil: str) -> None:
        pass
```

- `Rascunho`: encaminha o documento para moderação
- `Moderação`: permite a publicação quando o perfil é administrador
- `Publicado`: mantém o documento publicado
- Ao criar o próximo estado, passamos a referência para o mesmo documento

Aplicando ao nosso contexto - Execução

```
documento = Documento()
print(documento.estado)

documento.publicar("autor")
print(documento.estado)

documento.publicar("autor")
print(documento.estado)

documento.publicar("administrador")
print(documento.estado)

documento.publicar("autor")
print(documento.estado)
```

- O documento recebe chamadas ao mesmo método, mas o estado ativo determina o comportamento

Prós e contras do State

Prós

- Reúne o comportamento de cada estado em uma classe
- Reduz condicionais de estado espalhadas pelo Contexto
- Permite localizar as regras de cada estado com facilidade

Contras

- Aumenta a quantidade de classes
- Pode adicionar complexidade desnecessária a casos simples
- Estados podem depender de outros estados para realizar transições

Use:

- Quando o comportamento de um objeto depende do seu estado atual
- Quando as regras de comportamento e transição mudam com frequência
- Quando vários métodos repetem condicionais para verificar os mesmos estados



Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br